FLYRANK INTERNSHIP · FRONTEND AI ENGINEERING TRACK · FOUNDATIONS

React App Development with AI

Comprehensive Engineering Report: Architecture, Heuristic Quality Scoring, Model Simulation, and Rigorous Post-AI Code Refactoring in **PromptCraft Studio AI**.

CANDIDATE	TRACK & PHASE	AI DEVELOPMENT ENGINE	ASSIGNED WORKLOAD
KM Siam	**Frontend AI · Week 3**	**Google Antigravity**	**6 Hours · Foundations**

1. Executive Summary & Mentor Alignment

This project satisfies the **Week 3 Foundations** requirement to build a complete, production-ready React application independently using AI as an engineering co-pilot, following the architectural guidelines articulated by mentor **Ishak**. Rather than passively accepting raw AI output, the engineering methodology centered on an **explore-plan-code-verify** paradigm orchestrated within **Google Antigravity** (powered by Gemini 3.8 Flash High / Advanced Agentic Coding).

The result is **PromptCraft Studio AI**: an interactive, single-page application built in React 19 and Vite 8.3 that empowers engineers to compose dynamic prompts with live `{{variable}}` interpolation, evaluate prompt quality via heuristic static analysis (Clarity, Specificity, Guardrails), simulate streaming token output across frontier models, and monitor real-time FinOps API costs.

> **Mentorship Core Insight:** As emphasized by mentor Ishak, the value of an AI assistant lies not in autonomous typing, but in accelerating the translation of structured specifications into modular components while the engineer exercises strict gatekeeping over state flow, lifecycle boundaries, and edge-case resilience.

2. System Architecture: PromptCraft Studio AI

The application is architected around a 3-column responsive workbench layout ensuring zero layout shift and sub-millisecond local state synchronization:

1. Template Navigator (Left Column)

Instant search filtering by keyword or tags, domain category switching (*All, Coding, Compliance, Analysis, System*), dynamic variable count badges, cloning, deletion, and pre-seeded enterprise templates.

2. Editor & Dynamic Variable Binding (Center)

Monospace prompt template textarea, system persona input, automatic regex extraction of `{{placeholders}}`, reactive input fields, and real-time interpolated prompt preview with 1-click clipboard copying.

3. AI Quality Scorecard (Right Column)

Heuristic static analyzer scoring Clarity (35%), Specificity (35%), and Guardrails (30%) on a 0–100 scale, assigning quality tiers (*EXCELLENT, GOOD, MODERATE, POOR*) with actionable optimization recommendations.

4. Model Playground & FinOps Telemetry (Right Column)

Simulates token streaming (45ms interval) with abort control across Gemini 3.8 Flash High, Claude 3.7 Sonnet, and GPT-4o. Displays latency, token throughput, and exact USD transaction cost down to 6 decimals.

3. Technology Stack & Build Metrics

Component / Dependency	Version	Architectural Function	Status
React & ReactDOM	`19.3.1`	Core component rendering, concurrent lifecycle hooks, pure functional composition	VERIFIED
Vite	`8.3.6`	Next-generation bundler with Hot Module Replacement & lightning Rollup/Oxc builds	597MS BUILD

Component / Dependency	Version	Architectural Function	Status
Lucide React	`1.69.0`	Tree-shakeable enterprise icon library for visual affordances & status indicators	ACTIVE
Local Persistence	`Native Web Storage`	Persistent browser synchronization of user edits, clones, and custom templates	ZERO-LAG
FinOps Pricing Engine	`Custom Heuristic`	Deterministic token estimation (1.33x words) and multi-model USD cost matrix	CALIBRATED

4. AI Assistance Workflow & Prompting Methodology

Development was directed in **Google Antigravity** across five distinct phases using structured prompt engineering patterns (Context-Task-Constraint-Verification). The complete log of prompts is preserved in `PROMPTS.md`.

Phase	Antigravity Role	Prompts Directed	Output Produced
1. Architecture & Blueprint	Systems Architect	Component boundaries, layout constraints, and dependency specifications	`App.jsx` layout blueprint, `package.json`, `vite.config.js`
2. Algorithmic Logic	Algorithmic Engineer	Deterministic regex placeholder extraction, token math, FinOps pricing	`src/utils/promptParser.js`, `src/utils/evaluator.js`
3. Component Synthesis	UI/UX Specialist	Header, TemplateList, PromptEditor, VariableInputs, Scorecard, Playground	6 modular, accessible JSX components with dark-palette inline styling
4. State & Persistence	State Engineer	State lifting, template CRUD, duplicate logic, JSON export and import	Dynamic template switching, `localStorage` synchronization
5. Automated Testing	QA Automation Lead	Pure Node.js assert test suite covering 10 mission-critical paths	`tests/promptcraft.test.js` (10/10 passing tests)

5. Critical Post-AI Manual Refactorings (Code Review)

As documented in `REFACTORING.md`, human code review identified four critical areas where raw AI output required significant manual correction and re-architecture:

Refactoring 1: Streaming Timer Memory Leak & Template Switch Race Condition

The Flaw: The AI initiated `setInterval` for token simulation without storing the timer handle in a `useRef` and without clearing it upon template switching or unmount. Switching templates mid-stream produced concurrent runaway intervals that corrupted output telemetry.
The Fix: Introduced `streamTimerRef`, coupled with a cleanup effect in `useEffect([template?.id])` and an abort button that resets the generator state cleanly.

Refactoring 2: Regex Deduplication & Safe Placeholder Interpolation

The Flaw: The AI's initial regex returned duplicate entries when a placeholder (e.g., `{{input_data}}`) was referenced multiple times, causing duplicate input fields. Furthermore, unfilled variables were replaced with blank strings, destroying prompt context.
The Fix: Added `Set` deduplication to `extractVariables` and updated `interpolatePrompt` to preserve original placeholders until values are provided.

Refactoring 3: Schema Property Desynchronization (systemRole vs systemPersona)

The Flaw: Pre-seeded templates declared `systemRole`, while `App.jsx` and test assertions looked for `systemPersona`, causing blank inputs upon initial load and failing unit tests.
The Fix: Standardized on `systemRole` across all modules and implemented defensive fallback resolution: `template.systemRole || template.systemPersona || ''`.

Refactoring 4: Test Environment Decoupling (Pure ESM vs Bundler Pipeline)

The Flaw: AI suggested running tests importing raw JSX files directly in Node.js ESM mode, throwing `ERR_UNKNOWN_FILE_EXTENSION`.
The Fix: Decoupled logic tests into pure JavaScript executed natively by Node.js, while delegating JSX bundling verification to Vite's production build.

6. Verification Proof & Automated Test Execution

```
$ npm test

> flyrank-internship-front-end-ai-engineering@1.0.0 test
> node tests/promptcraft.test.js
```

```
🚀 Starting PromptCraft Studio AI Automated Test Suite...

--- 1. Variable Extraction & Template Interpolation ---
  ✅ PASS: extractVariables extracts multiple unique curly-brace placeholders
  ✅ PASS: extractVariables handles empty strings and prompts without placeholders
  ✅ PASS: interpolatePrompt replaces matched variables and preserves unmatched ones
  ✅ PASS: interpolatePrompt keeps placeholder if variable key is missing

--- 2. Token Estimation & FinOps Cost Calculation ---
  ✅ PASS: estimateTokens calculates approximate token counts accurately
  ✅ PASS: estimateTokens handles null and empty text gracefully
  ✅ PASS: calculateEstimatedCost computes accurate USD costs across models

--- 3. Static Quality Evaluation Heuristics ---
  ✅ PASS: evaluatePromptQuality assigns high scores to well-structured prompts
  ✅ PASS: evaluatePromptQuality identifies weak prompts and provides actionable suggestions

--- 4. Pre-seeded Domain Templates Integrity ---
  ✅ PASS: all default templates have valid schema and structure

========================================
Summary: 10/10 tests passed.
========================================
🎉 All automated tests passed successfully!
```